中国研究生操作系统开源创新大赛

项目说明书 · 内核优化分册

作品名称：面向智能机器人的 AIOS

参赛队伍：Xenoblade

完成日期：2026 年 10 月

目 录

第 1 章 优化总览与评测口径 **1**

1.1 评测平台与口径 1

1.2 优化之前的总体画像 1

1.3 优化清单 3

1.4 优化之后的总体结果 4

1.5 结论 4

第 2 章 系统调用入口与返回路径 **5**

2.1 问题定位：`getpid` 是最纯的探针 5

2.2 分段归因 5

2.3 根因：每次返回用户态都刷指令缓存 6

2.4 改法：把刷新收敛到少数事件上 6

2.5 实测 6

2.6 剩下的固定成本：为什么停手 7

2.7 小结 7

第 3 章 进程创建、exec 与上下文切换 **8**

3.1 优化前的账 8

3.2 `fork` 的全路径归因 8

3.2.1 已落地之一：取消每个叶子的第二次页表走查 9

3.2.2 已落地之二：所有者发布去重的复杂度 9

3.2.3 已落地之三：地址空间退休不再逐页摘除 9

3.3 `exec`：从 32 ms 到 4.5 ms 9

3.4 上下文切换：归因清楚了，但没动 10

3.5 实测 11

第 4 章 缺页与地址空间 **12**

4.1 第一层归因：87.7% 在 `prepare` 12

4.2 第一刀：COW 页索引不再每次全量重建 12

4.3 第二刀：内存操作从逐字节改字长 13

4.4 第三刀：顺序缺页预取 14

4.4.1 窗口大小：做成旋钮再扫描，不拍常数 15

4.5 第四刀：窗口内的帧整块分配 16

4.6 第五刀：页索引按 run 批量登记 16

4.7 实测总表 17

4.8 剩下的每页预算 18

第 5 章 文件读路径 **19**

5.1 第一条线索：省掉一次拷贝 19

5.2 第二条线索：用户内存准备整段化 19

5.3 第三条线索：瓶颈在 `close`，不在读 20

5.3.1 定位链：从 `close` 一直到 `atime` 20

5.3.2 改法：按 Linux 的 `relatime` 语义处理访问时间 20
5.3.3 边界：这一刀不改善“读”本身 21
5.4 回到读路径：fresh 缓冲的三段拆解 21
5.5 实测与边界 22
5.6 剩余空间 22
第 6 章 存储写回与元数据 **23**
6.1 优化前 23
6.2 第一刀：脏页按连续 run 批量写回 23
6.3 第二刀：页缓存容量 23
6.4 第三刀：`close` 路径不再做同步元数据提交 24
6.5 与 Linux 的整体对照 24
6.6 未完成：屏障语义下推 24
第 7 章 启动时间 **25**
7.1 口径：把启动切成三段 25
7.2 本轮改动：把固定阻塞等待改成带截止时间的轮询 25
7.3 剩下的静默块 26
7.4 小结 26
第 8 章 负结果与未完成项 **27**
8.1 被数据否掉的假设 27
8.2 已经归因清楚、但本轮没有改的项 27
8.3 一条尚未定位清楚的问题 28
8.4 提交清单 29
第 9 章 复现与验收 **30**
9.1 平台与镜像 30
9.2 探针清单 31
9.3 命令 31
9.4 验收判据 32
9.5 与主报告的关系 32

第 1 章　优化总览与评测口径

本分册只讲内核层。应用层的视觉推理链路（JPEG 硬解、VPSS 几何变换、TPU 推理、 检测后处理与追球控制）另见《第二部分：追球推理链路》，两者共用同一块板、同一份 用户态 ELF，互不重叠。

内核层的工作目标很具体：项目立项时的横向对比显示，同一块板、同一份静态 riscv64 ELF 上，StarryOS 在“每次系统调用、每次进程创建、每次缺页、每次文件读” 这些**固定开销**上比厂商 Linux 贵一到两个数量级，而在内存带宽、大块顺序写、 冷启动这些**带宽或总量型**指标上并不落后。因此这一阶段不做功能补全，而是沿着“把每次都要付的固定成本降下来”这一条主线，逐个热点做归因、改机制、再实测。

1.1　评测平台与口径

平台与对照物固定如下，本分册所有数字都在这一组条件下取得：

项	内容
板卡	AKA-00-SG2002（LicheeRV Nano 血统），CPU 为 C906（riscv64，单应用核），内存 236 MiB
对照系统	厂商 Linux 5.10.4，卡上 `linux.sd`，同板同卡同一次上电周期内交替启动
用户态	同一份静态链接 riscv64 ELF。所有探针都是自研 C 程序，两边编译同一份源码
计时	板端 `rdtime`（时基 1 MHz）+ 主机侧串口逐行时间戳；微基准取 20–100 轮中位
口径	每个指标三列：Linux 实测／我们优化之前／我们优化之后

表 1: 评测平台与对照口径

有三条读表纪律，先说清楚，否则数字容易被误读：

1. **同板同 ELF 才算对照。** 所有“相对 Linux 的倍数”都取自本板实测的厂商内核， 不引用其它论文或其它板卡的经验值。Linux 侧缺测的指标用 **未测** 标出， 不拿别处的数字补位。
2. **冷启动受控 A/B。** 内核“优化前/后”的对比尽量在**同一次上电、同一份探针、 交替顺序**下做（例如 `opt7` → `opt10` 两两交替各测两遍取均值），以抵消“启动后 第一次大分配明显更快”这类时间漂移。
3. **探针本身有代价。** 热路径上的分段计时是每次系统调用多 12 次取时间戳， 实测平白增加约 1.2 µs。凡是“优化前/后”的对照数字，探针都已撤掉；带探针的 中间过程只在归因小节里出现，并注明口径。

1.2　优化之前的总体画像

优化起点（2026-10-04 的 `ours` 基线镜像）与 Linux 的对照如下。这张表决定了后面 每一章的优先级：凡是差距在一个数量级以上的，都是“每个操作都要付的固定成本”；凡是基本持平的，都是带宽型或总量型指标。

指标	Linux 5.10.4	我们（优化前）	差距
`getpid`（空系统调用）	422.5 ns	10 560 ns	25.0×
`pipe_wr`（4 KiB 往返）	4 489 ns	46 819 ns	10.4×
管道 ping-pong（含一次切换）	26.0 µs	445.7 µs	17.1×
`fork+exit+waitpid`	338.1 µs	7 497 µs	22.2×
`fork+execve+waitpid`	2 407 µs	43 537 µs	18.1×
上下文切换	10.97 µs	226.8 µs	20.7×
64 MiB 匿名首触（默认 4 KiB）	7 899 ns/页	1 302 123 ns/页	165×
ext4 `open+read`(1 KiB)+`close`	42 µs	10 320 µs	246×
8 MiB 顺序写 + `fsync`	790 ms	7 961 ms	10.1×
缓存命中大读（缓冲已触碰）	671.6 MB/s	374 MB/s	1.80×
缓存命中大读（每轮新缓冲）	271.1 MB/s	50.2 MB/s	5.40×
内存 `memcpy` / 顺序读带宽	1.024 / 1.488 GB/s	0.984 / 1.420 GB/s	1.04× / 1.05×
端到端冷启动（`bootm`→提示符）	33.2 s	13.21 s	我方更快 2.5×

表 2: 优化起点的横向对照。前九行是“每个操作都要付的固定成本”，差距都在一个数量级以上；末四行是带宽型或总量型指标，本来就不落后

这张表给出三点判断，后续九章都围绕它们展开：

1. **系统调用入口是最大的单点。**一个什么都不做的 `getpid` 要 10.56 µs，是同板 Linux 的 25 倍。它的处理体只取一个已在内存里的进程号，所以这 10.56 µs 几乎全部 是“进入内核、返回用户态”这条固定路径。这也是第 2 章的对象。
2. **调度与进程生命周期的固定开销排在第二。**`fork+exit` 22 倍、上下文切换 20 倍、管道往返 17 倍、exec 18 倍，全都在同一个量级上。第 3 章把这条链路拆成“页表复制 / 首次入队 / 子进程析构 / 每次切换”四段分别处理。
3. **内存的“每页固定成本”比内存带宽问题严重得多。** 64 MiB 首触慢 165 倍，而 `memcpy` 带宽只差 4%。说明缺的不是带宽，而是**每页一次事务**的簿记开销。 第 4 章与第 5 章处理这一条（缺页路径、文件读路径）。

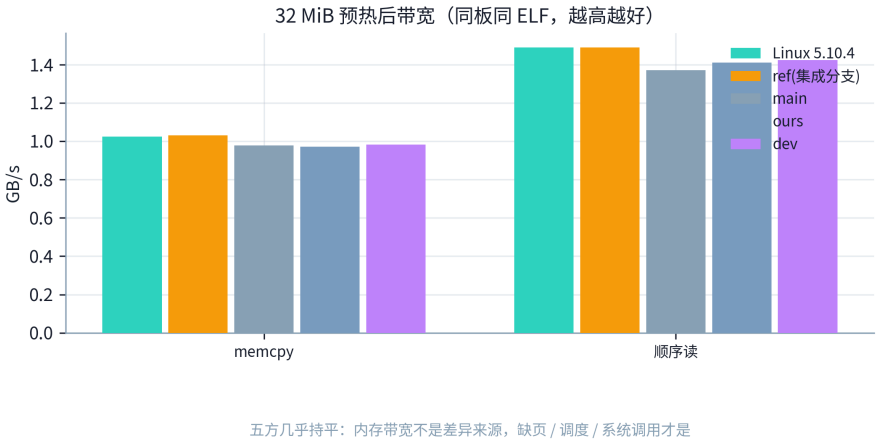


图 1: 五方对照下的内存带宽。`memcpy` 与顺序读在各方之间只差几个百分点——它说明本板的差距不在内存子系统，而在“每次操作要付多少固定成本”

还有两条“看起来很小、实际很贵”的发现，分别对应第 6 章和第 7 章：ext4 上每次 `open+read+close` 要付一次约 10 ms 的同步日志提交（Linux 只在内存里改 atime），以及启动期一个固定 2 s 的 DHCP 阻塞等待。

1.3　优化清单

下表是本阶段实际落地的改动，按子系统分组。每条都对应一个独立提交，且都满足 两个约束：**不与参考实现（报告团队的集成分支 `ref`）使用同一手段**，以及 **有“优化前/优化后”的同板 A/B 数据**。

编号	改动	主要手段（与 `ref` 的差异）
B1	返回用户态不再每次刷 I-cache	把 `fence.i` 收敛到“用户映像可能陈旧”的少数事件上；`ref` 未做
A1	COW 页索引不再每次全量重建	去掉每次插入的两遍 O(n) 存活计数，墓碑改在容量增长路径上摊还裁剪
A3a	`memcpy`/`memset` 从逐字节改字长循环	消灭 64 位平台上的字节循环；`ref` 走的是 `dc zva`（aarch64 专属），riscv64 不可用
A3b	顺序缺页预取窗口（64 KiB）	严格升序且私有匿名的缺页按窗批量物化；`ref` 是默认 2 MiB 大页，我们保持 4 KiB 叶子
A3c	窗口大小运行期旋钮 + 短流爬坡	旋钮扫描定值 64 页，并按连续命中次数爬坡，避免短流被整窗放大
A3d	窗口内帧整块分配 + 子租约	一次 `alloc_frame` 再切 4 KiB 子租约，块命中率 99.97%
A3d′	COW 页索引按 run 批量登记	一次加锁、一次定位、一次整段落位，且全有或全无；`ref` 未做
CC1	地址空间退休不再逐页 unmap	整段交给批量路径；子进程析构 3.57 ms → 0.41 ms
H1	exec 只物化可写段 + 按 run 批量 pin	static-PIE 只读段改按需缺页；与 `ref` 的 `MADV_POPULATE_*` 路线完全不同
H2	页缓存 pin 住后直接拷给调用方	去掉中间的 scratch 中转，少一次全量拷贝
H3	uaccess 整段快路径 / 整段 `populate_area`	riscv64 无硬件探测，改成按段判定并整段物化
H5	`close` 不再无条件同步写 atime	按 Linux 语义做 `relatime`，并用单调时钟而非墙上时钟判定
D1	脏页按连续 run 合并写回	8 MiB 从 2048 次 4 KiB `write_at` 变成少数几次大块写
D2	页缓存上限 512 → 4096 页	只改容量参数，不改回收策略
E1	启动期 DHCP 等待 2 s → 300 ms	把固定阻塞等待改成带截止时间的轮询

表 3: 本阶段落地的内核改动。编号沿用内部优化清单，便于与原始测量记录对照

1.4　优化之后的总体结果

指标	Linux	优化前	优化后	优化后/ Linux
`getpid`（空系统调用）	422.5 ns	10560 ns	1631 ns	3.9×
`pipe_wr`（4 KiB 往返）	4489.2 ns	46819 ns	12576 ns	2.8×
管道 ping-pong	26 µs	445.7 µs	421.1 µs	16.2×
`fork`+`exit`+`waitpid`	338.1 µs	7497 µs	6461 µs	19.1×
`fork`+`execve`+`waitpid`	2407 µs	43537 µs	22359 µs	9.3×
上下文切换	10.97 µs	226.8 µs	220.4 µs	20.1×
双线程 `sched_yield` 交接	6.9 µs	171 µs	112.5 µs	16.3×
64 MiB 匿名首触（默认 4 KiB）	7899 ns/页	1302123 ns/页	44120 ns/页	5.6×
64 MiB 首触（显式大页）	5683 ns/页	22216 ns/页	8014 ns/页	1.41×
ext4 `open`+`read`(1 KiB)+`close`	42 µs	10320 µs	454 µs	10.8×
8 MiB 顺序写 + `fsync`	790 ms	7961 ms	931 ms	1.18×
8 MiB 页缓存顺序写	34 MiB/s	2.41 MiB/s	42.43 MiB/s	我方更快
缓存命中读（warm）	671.6 MB/s	374 MB/s	474 MB/s	1.42×
缓存命中读（fresh）	271.1 MB/s	50.2 MB/s	62.8 MB/s	4.32×
单次大读 803 KB	277.2 MB/s	60.9 MB/s	76.2 MB/s	3.64×
冷启动到可交互 shell	33.2 s †	12.3 s	10.8 s	我方更快

表 4: 优化前后总表。Linux 列为同板实测；“优化后/Linux”小于 1 表示我方更快。上下文切换与管道往返本轮只做了归因、未改行为，故仍落后一个数量级（第 3 章）；末行两侧的启动锚点不同（见第 7 章的说明），此处只作量级参考

1.5　结论

把这张总表读完，可以得到四条结论：

1. **“每次调用都要付”的固定成本已经系统性下降，但没有到齐**。空系统调用从 25.0 倍收到 3.9 倍，4 KiB 管道往返从 10.4 倍收到 2.8 倍，exec 从 18.1 倍收到 9.3 倍，ext4 `open`+`read`+`close` 从 246 倍 收到 10.8 倍。
2. **缺页这条最深的坑基本填平**。64 MiB 默认路径从 1302123 ns/页 降到 44120 ns/页（**29.5**×），与 Linux 的差距从 165 倍收到 5.6 倍，并且 成本已经**不再随区域大小增长**。四条改动之间彼此独立，任何一条都可以单独回退。
3. **带宽型指标本来就领先或持平，优化后仍然领先**。 页缓存顺序写从 2.41 MiB/s 提到 42.43 MiB/s（Linux 34 MiB/s）， 冷启动 10.8 s 对 Linux 33.2 s。
4. **剩下的差距集中在两处，且都已被精确定位但没有动**。 一是上下文切换（220.4 µs 对 10.97 µs），根因是 EEVDF 调度器热路径上十几处 1–15 µs 的固定开销叠加 （第 3 章）；二是文件读的每页固定成本（fresh 仍差 62.8 对 271.1 MB/s），根因是每页的 `MappingSlot`/rmap 发布与 `PageObject` 堆分配（第 5 章）。两者都需要改动核心不变式，本轮只做了归因、没有改行为，详见第 8 章。

第 2 章　系统调用入口与返回路径

系统调用入口是用户态与内核之间唯一的门。无论一次调用只是取回一个进程号，还是发起一次文件读写，都要先支付同一段固定成本：陷入、保存现场、按调用号分发、 执行处理体、恢复现场、返回用户态。调用体越轻，这段路径在总耗时里的占比越高。本板是单应用核，进程间通信、文件访问、模型加载都建立在这条路径之上，所以它是 “固定成本”里最该先动的一处。

2.1　问题定位：`getpid` 是最纯的探针

自研探针 `syscost2` 测量九个系统调用的单次开销。优化之前，`getpid` 需要 10560 ns，同板 Linux 只要 422.5 ns，相差 25 倍。更有说服力的 是另一组对照：`invalid_syscall`（故意传入非法调用号，内核里只有“立即返回 `ENOSYS`”这一条路）与 `getpid` 一样慢。处理体本身只从当前线程结构里读一个 字段，它不可能值 10 µs，所以这 10 µs 几乎全部属于“进入内核再返回”这条路径。

这组数据同时否掉了两种常见的猜测：不是“某个系统调用实现得差”，也不是“某个子系统慢”。九个调用（`getpid`、`getppid`、`clock_gettime`、`nanosleep(0)`、 `getrusage`、`write` 到 `/dev/null`、`getitimer`、`setitimer(0)`、`mmap+munmap`）在优化前呈同一条水平线，说明它们共享同一笔固定成本。

2.2　分段归因

给陷阱循环挂上分段计时后，一次 `getpid` 的构成如下（`syscost2` 口径， 单位 ns，同一次测量的增量平均）：

段	ns/次	说明
陷阱循环整体 `iter`	10 687	用户态进入 → 返回值写回
└ 进入/返回用户态 `enter`	5 558	架构层的一整趟往返
└ 分派与函数体 `handle`	2 654	按调用号查表 + `getpid` 本体
└ ptrace 前置检查 `pre`	264	每次进入都判一次“是否被跟踪”
└ 返回后处理 `post`	439	信号与返回前记账
其它（用户态侧循环、统计口径）	1 772	探针自身与两侧差值

表 5: `getpid` 的分段归因（优化前）。处理体只占 2.65 µs，其余都是路径成本

把最大的一段 `enter` 再拆开，答案就落在一个指令上：

`enter` 内部段	ns/次	说明
`fence.i`	2 172	每次返回用户态都执行一次指令缓存失效
真正的往返 `enter_user`	2 150	寄存器保存/恢复、特权级切换
入口校验	267	参数与状态检查
合计	4 589	其余为零头

表 6: 返回用户态路径的内部分段（301 449 次进用户态的增量平均）

2.3　根因：每次返回用户态都刷指令缓存

`fence.i` 在 riscv64 上的语义是“让本核此后取到的指令都来自最新的指令存储”。内核在**修改过用户映像的代码字节、或者把一个页变成可执行**之后，必须刷一次，否则用户态可能执行到陈旧指令。这是必要的语义。

问题在于实现把它放到了**每一次返回用户态**上。代价有两层：

- 它自己就值 2 172 ns，占当时 `enter` 的 39%；
- 它把此后**整条内核路径的指令缓存打冷**——下一次系统调用再进内核时，内核代码要重新从内存里取指令。这也解释了为什么 `handle` 段在带探针时 看起来有 2.65 µs，而“处理体本身什么都不做”。

而内核本来就已经在真正需要的地方刷：`PageObject::prepare_executable_mapping()` 在**任何一条把页变成可执行的装页表路径**上都会刷（共 11 处调用点），`AddrSpace::sync_modified_text()` 在 ptrace 改写正文后也会刷。也就是说，返回路径上的这一次是纯冗余。

2.4　改法：把刷新收敛到少数事件上

改动只做一件事：**默认不刷，只在用户映像可能陈旧时刷一次**。触发条件收成四类：

- `execve` 装载了新映像；
- `mmap` 或 `mprotect` 把一段内存变成可执行（`PROT_EXEC`）；
- 切换到另一个地址空间（换页表基址）；
- 显式写 `/proc/icache_flush`。

为了能随时回退，`/proc/icache_flush` 同时是一个运行期旋钮：写 1 恢复“每次返回都刷”的旧行为。这段语义动的是正确性边界，所以验证上做了三层：宿主单测、板端全项 harness，以及 78 轮多二进制 `exec` 压力（反复装载不同 可执行文件、每次校验输出）。

2.5　实测

指标	Linux 5.10.4	优化前	优化后
`getpid`（空系统调用）	422.5 ns	10560 ns	1631 ns
`pipe_wr`（4 KiB 往返）	4489.2 ns	46819 ns	12576 ns
双线程 `sched_yield` 交接	6.9 µs	171 µs	112.5 µs
管道 ping-pong	26 µs	445.7 µs	421.1 µs
上下文切换	10.97 µs	226.8 µs	220.4 µs
`memcpy` / 顺序读带宽	1.024 / 1.488 GB/s	0.984 / 1.420 GB/s	0.981 / 1.428 GB/s

表 7: 返回路径瘦身的实测。前三行是“调用体很轻”的指标，收益最直接；后三行主要由调度与内存主导，与归因一致

其中 `getpid` 从 25.0 倍收到 3.9 倍，`pipe_wr` 从 10.4 倍收到 2.8 倍。`sched_yield` 交接下降 31%，其余几项没有可观变化。

有两个口径必须说清楚，否则数字会被误读：

1. **带探针时会低估收益**。 探针是每次系统调用多 12 次取时间戳，实测平白增加 约 1.2 µs。上表是**撤掉热路径探针之后**的干净数字；同一镜像在带 5 个系统调用 探针时，`getpid` 显示为 3 050 ns。也就是说这项优化的真实收益是 `getpid` 6.5 倍、 `pipe_wr` 3.7 倍。
2. **`getpid` 的复测区间是 1233–1631 ns**。 同一镜像多次复测 会落在这一区间内，取最差值入表；最好的 1233 ns 对应 2.9 倍。

2.6　剩下的固定成本：为什么停手

撤掉探针后再用 `syscost2` 与 `sa1` 细拆， `getpid` 剩下的 3.05 µs 是：

段	ns/次	说明
返回用户态往返	1 070	架构地板：已无可摘除的冗余
循环内其它	560	采样与记账
分派与函数体	360	查表 + `getpid` 本体
ptrace 前置 + 返回前信号检查	230	每次陷阱都查一遍的开关
用户侧	840	探针循环本身

表 8: 返回路径瘦身之后 `getpid` 的剩余构成。已经没有单项超过 1.1 µs

剩下的都是同一类东西：**每次陷阱都要走一遍的开关判断**——单步标志 91 ns、 ptrace 速查 102 ns、返回前工作 149 ns、seccomp 88 ns、trace 101 ns。要让它们 再快，得把这些分散的布尔判断合并成一次位图检查，属于返回路径的结构性重构，收益上限约 1.5 µs（相对 Linux 的 422.5 ns 仍是 2–3 倍区间），风险与收益不成比例， 因此本轮到此为止，把剩余空间记入第 8 章的未完成项。

2.7　小结

这一章处理的是一条**看起来最不像瓶颈**的路径：它没有热点函数，九个系统调用 平行地慢，历史上多次归因都指向“没有单一问题”。分段计时的价值在这里体现得最清楚 ——把 `enter` 拆到指令级之后，一笔 2.17 µs 的冗余刷新浮出水面，删掉它同时 恢复了被它打冷的指令缓存，于是 `getpid` 一次下降 6.5 倍。

第 3 章　进程创建、exec 与上下文切换

这一章处理三个互相关联的指标：`fork+exit+waitpid`、`fork+execve+waitpid` 与一次上下文切换。优化之前它们分别落后同板 Linux 22.2 倍、18.1 倍与 20.7 倍。 三者共享同一批底层结构（页表复制、任务入队、地址空间退休、EEVDF 就绪队列），所以这一章的做法是先把整条链路拆到段，再挑出**纯算法、无行为变化**的部分动手。

3.1　优化前的账

指标	Linux 5.10.4	优化前	差距
`fork` + `exit` + `waitpid`	338.1 µs	7497 µs	22.2×
`fork` + `execve` + `waitpid`	2407 µs	43537 µs	18.1×
上下文切换	10.97 µs	226.8 µs	20.7×
双线程 `sched_yield` 交接	6.9 µs	171 µs	24.8×

表 9: 进程与调度面的优化起点。四项同源，差距都在一个数量级到两个数量级之间

3.2　`fork` 的全路径归因

`fork_exit` 优化前是 7 497 µs，此前只有一个总数。给内核侧（`mm/fault_attrib.rs`） 与任务侧（`components/ax-task/src/diag.rs`）各加一组分段计数器，全部渲染在 `/proc/fault_attrib`，用“工作负载前后的快照之差”取每次调用的平均值，得到的账是：

层	每次	明细（µs）
`fork()` 系统调用	4 418	try_clone 1 528 / 首次入队激活 1 724 / 建线程 591 / fd+FS+线程 194 / 信号+映像+进程台账 197 / 发布 102 / pid+cgroup+命名空间 17
子进程 `_exit()`	1 013	fd 271 / 进程台账 274 / 僵尸发布+信号+唤醒 481 / 退地址空间 2.4
子地址空间回收	1 459	**不在** `fork_exit` 的墙钟里——由内存回收线程异步完成，每次 fork+exit 约 1.4 ms CPU
上下文切换	226×11–14 次	`fork_exit` 里约 2.6–3.2 ms 花在这里

表 10: `fork+exit` 的分段账（优化前，单位 µs/次）。这张表直接改写了优化优先级：最大的两块分别是“首次入队触发的重调度”与“上下文切换次数”

归因给出两条决定性判据，它们把后续方向定死了：

1. **首次入队激活的 1.7 ms 全在守卫析构。** 把它拆成三段后：入队 44 µs、收尾 0.6 µs、定时器 10.5 µs，剩下的 1 852 µs 全在 `RuntimeIrqGuard` 的析构里 ——激活时 `finish_owner_enqueue` 已经请求重调度，守卫析构于是直接进入 “从 IRQ 守卫退出时挑下一个任务”。
2. **这 1.7 ms 不是“子进程占用了 CPU”。** 让子进程在 `fork` 返回后先忙等 T ms 再退出，父侧 `fork()` 的耗时完全不随 T 增长（子进程快速退出 5 527 µs， 忙等 10 ms 后退出 4 529 µs）。所以它是**切换本身**的成本：切到一个全新任务 再切回来。

顺带否掉了两个假设：硬件 ASID 是可用的（`asid_tag_capacity=65536`，已经处于 标签模式，不存在“每次切换都全量刷 TLB”），以及子地址空间析构并不在 `fork_exit` 的关键路径上。

3.2.1　已落地之一：取消每个叶子的第二次页表走查

`occupied_leaf_records()` 本来已经拿到页表遍历时的原始 PTE，却对每个叶子再调一次 `pt.query()` 复核。改成复用遍历结果后，单个叶子的克隆成本从 99 µs 降到 82 µs。

3.2.2　已落地之二：所有者发布去重的复杂度

`publish_prepared_pte_owners()` 用线性 `Vec::contains` 检测重复键。批量填充一次要发布几百个所有者，代价是 $O(n^2)$。换成 `BTreeSet` 后，这段从 120 µs 降到 99 µs。

这两处纯算法改动合计让 `try_clone` 从 1 747 µs 降到 1 528 µs、`fork()` 系统调用 从 4 739 µs 降到 4 418 µs。

3.2.3　已落地之三：地址空间退休不再逐页摘除

子进程退出时要退休整个地址空间。原实现逐页 `unmap`，每次都要走查页表、再走查一次页表、更新地址空间账目。改成整段交给批量路径后：

项	优化前	优化后
子地址空间退休 `exit_clear`	3570 µs	**410 µs**（8.7×）
`fork_exit` 端到端	7497 µs	6461 µs

表 11: 退休路径批量化。这 3.2 ms 的收益不在 `fork_exit` 的墙钟里（它由回收线程异步做），但直接减少了每次 fork+exit 的 CPU 占用

3.3　`exec`：从 32 ms 到 4.5 ms

`fork+execve` 里，`execve` 的内核侧成本优化前是 32.3 ms。分段后 88% 落在一处：

段	ms/次	说明
`exec_load`	31.7	装载映像
└ `exec_populate`	28.3	**88%**：static-PIE 把每个 `PT_LOAD` 段整段物化
└ `exec_map` / `exec_reloc` / `exec_parse`	19.5	嵌套计数：段映射含上面的物化；重定位 0.56、解析 0.05
`exec_stack`	1.13	建立用户栈
`exec_install`	0.25	安装新映像

表 12: `execve` 的分段（优化前）。重定位与解析都不是问题，问题在页物化

再往物化里挖，得到三条关键的否证：`file_pop_*`（文件后端的物化探针）**全程为 0**，说明段映射走的是 COW 而不是共享文件映射；块层读数在一次 51 轮的 exec 压力前后 **增量为零**，说明 exec 完全不读 SD 卡；因此每页 120 µs 的成本是“一次帧分配 + 三次 4 KiB 拷贝（页缓存 → scratch → 缓冲 → 新帧）+ `PageObject` 与 COW 索引 + 装页表”。

据此做了三刀：

1. **文件后端按连续 run 批量处理**。物化时把“逐页一次 `io_lock`”改成 `CachedFile::pin_pages_run`（整段一次锁）、`FilePageDomain::reserve_pages_run`（整段一次域锁），并把每段一次的全表 `prune_stale` 收成一次。

2. **只物化带写权限的段**。 static-PIE 的重定位只写 `.got` / `.data.rel.ro` / `.bss`，只读段完全可以按需缺页——Linux 也是惰性的。改成只整段物化 `PT_LOAD` 里带 `WRITE` 的那些，只读段交给缺页路径。
3. **单页缺页不再每次跑 O(表长) 的清理**。 `reserve_page` 原来每次都要扫一遍 陈旧条目表；改成“命中陈旧条目才清 + 每 512 次兜底”，单页缺页因此每页少一次 全表扫描。

实测：`exec_load` 从 31.7 ms 降到 4.5 ms，其中 `exec_populate` 从 28.3 ms 降到 1.2 ms；exec 内部的缺页数从约 200 页/次降到 79 页/次；`fork_exec_exit` 从 43537 µs 降到 22359 µs（−49%）。

值得一提的是，这一步与参考实现的路线完全不同：`ref` 用 `MADV_POPULATE_*` 把 整段一次性铺好，我们是**反过来减少要铺的页**，因此两者的收益不会互相抵消。

3.4　上下文切换：归因清楚了，但没动

一次 `ctx_switch` 优化前是 226.8 µs，Linux 10.97 µs。它不是一个“切换指令”的耗时，而是“管道写/读系统调用 + 阻塞 + 唤醒 + 一次切换”的整包成本。把它拆开：

段	µs/次	说明
管道写（含唤醒）	61	其中唤醒 49：选中等待者 0.35、交付 43–47
└ 入队：EEVDF 就绪树插入	13.6	红黑树插入与虚拟时间更新
└ 入队：其余六段	11.7	实体克隆、`settle_current`、`push_class` 等
调度入口	2.1×4–6 次	其中约 1/3 是空转：进去发现不需要切换，仍付全额
调度决策	15.6	EEVDF 的选取与虚拟时间维护
裸交接（`sw_*`）	5	地址空间激活 2.0 + CPU 绑定 0.4 + 汇编交接 2.2

表 13: 一次上下文切换的账。**没有单一热点**：十几处 1–15 µs 的固定开销叠加，其中“每次切换要进 4–6 次调度器、1/3 是空转”是可以立刻回收的一块

这一章在此止步，原因是它已经不是“修一处 bug”的规模：要逼近 Linux 的 10.2 µs，必须重构 EEVDF 热路径（空转入口前置判定、入队合并快照、收紧 `settle_current` 与虚拟时间更新），三条都动调度器不变式。本轮先把归因做完——这是后续任何改动的 前提——并把候选方案记入第 8 章。

顺带记录一条**负结果**，避免以后重复尝试：把“首次入队立刻抢占”改成 Linux 的 `WF_FORK` 语义（用运行期旋钮在同一镜像上做 A/B），只值 `fork_exit` 0.4%、`fork_exec_exit` 0.9% 的改善。原因是那 1.7 ms 里子进程该做的工作本来就要付， 抑制抢占只是把时间从 `fork` 挪到 `waitpid`。

3.5　实测

指标	Linux 5.10.4	优化前	优化后
`fork` + `exit` + `waitpid`	338.1 µs	7497 µs	6461 µs
`fork` + `execve` + `waitpid`	2407 µs	43537 µs	22359 µs
└ 其中 execve 内核侧	未测	32.3 ms	**4.5 ms**
上下文切换	10.97 µs	226.8 µs	220.4 µs
双线程 `sched_yield` 交接	6.9 µs	171 µs	112.5 µs
子地址空间退休	未测	3570 µs	410 µs

表 14: 进程与调度面的实测。`fork_exec_exit` 收掉 49%，`fork_exit` 收掉 14%；上下文切换本身本轮只做了归因，因此基本未动

`fork_exit` 只降了 14%，看起来与前面几处改动不成比例。原因在归因表里已经写明： 它剩下的时间主要不是页表复制，而是“切到一个新任务再切回来”的 11–14 次上下文切换，以及 EEVDF 热路径上的固定开销。因此 `fork` 的下一步与上下文切换是同一件事， 两者必须一起做。

第 4 章　缺页与地址空间

缺页是这块板上最深的坑。“私有匿名首触”（`mmap` 一块新内存然后逐页写入）在优化前 落后同板 Linux 165 倍，而且**成本随区域大小线性膨胀**：1 MiB 档 154 µs/页、64 MiB 档 1 302 µs/页。膨胀这个特征很关键——它说明问题不在内存带宽（`memcpy` 实测只差 4%），而在**每页一次的事务**里有什么东西是 O(n) 的。

这一章按“先把账算清、再逐项拿掉”的顺序组织：先用分段计账把 1.2 ms 拆到 8 段， 再依次拿掉页索引的 O(n)、字节级的内存操作、每页一次的事务、以及散落的帧分配。四刀彼此正交，任何一刀都可以单独回退。

4.1　第一层归因：87.7% 在 `prepare`

新增 `/proc/fault_attrib` 与对应探针，对同一段 64 MiB 分别测“默认 4 KiB 路径”与 “显式大页”：

阶段	默认路径 ns/页	占比	大页路径 ns/缺页	说明
`plan`	13 889	1.1%	14 101	VMA 查找与计划
`prepare`	**1 093 185**	**87.7%**	548 239	物化：分配帧、建对象、登记索引
`apply`	85 639	6.9%	73 451	装页表与发布
`tlb`	837	**0.07%**	682	TLB 失效——**不是瓶颈**
`mmu_cache`	669	0.05%	626	缓存维护
合计	1 246 183	100%	670 345	按页计

表 15: 缺页分段（优化前，64 MiB）。`tlb` 只占 0.07%，直接否掉了“缺页慢是因为刷 TLB”这个常见猜测再把占了 87.7% 的 `prepare` 拆开，答案落到一处数据结构上：

`prepare` 内部	ns/页	说明
`materialize` 小计	757 000	—
└ `frame_alloc`	15 000	全局页帧分配器
└ `frame_zero`	28 000	4 KiB 清零
└ `page_object`	21 000	建页对象
└ `pending_insert`	**719 039**	**COW 页索引插入，占整条路径的 58%**

表 16: `prepare` 的内部构成（优化前）。真正的问题不是分配、不是清零，而是每页一次的索引插入

4.2　第一刀：COW 页索引不再每次全量重建

`CowPageIndex` 是一个**按物理帧地址排序的 `Vec`**，用来回答“这个物理帧属于哪个 页对象”。原实现的问题在每次插入都要做两件 O(n) 的事：

- `insert_reservation_capacity()` 与 `rebuild_for_insert()` 各扫一遍**整个索引** 统计存活条目，每一项都是一次 `Arc::strong_count` 原子读（在大区域上必然缓存缺失）；
- 有序 `Vec` 的插入本身要把后面的条目整体搬走。

两者叠加，整条路径退化成 $O(n^2)$，成本随区域线性膨胀。

改法是纯数据结构层面的，不改任何语义：有富余容量时直接插入、跳过存活计数；已过期的弱引用条目改在**容量增长路径上摊还清理**；重叠检查只对存活条目生效。

指标	Linux 5.10.4	优化前	优化后
64 MiB 默认首触（探针）	7899 ns/页	1302123 ns/页	**162213 ns/页**
`pending_insert`	**未测**	719039 ns	**18461 ns**（39×）
标准 harness 32 MiB 首触	**未测**	10.54 s	**2.72 s**

表 17: 第一刀的效果。索引插入降了 39 倍，整条路径降了 8 倍，并且**成本不再随区域膨胀**

分档看“膨胀消失”这件事最直观（ns/页）：

区域	优化前	优化后	优化后/优化前
1 MiB	154179	168918	1.1×
4 MiB	307024	151250	2.0×
16 MiB	587627	153875	3.8×
64 MiB	1302123	162213	**8.0**×

表 18: 优化前每页成本随区域线性上升（154 µs → 1 302 µs）；优化后四条基本持平（151–169 µs）。区域越大收益越大

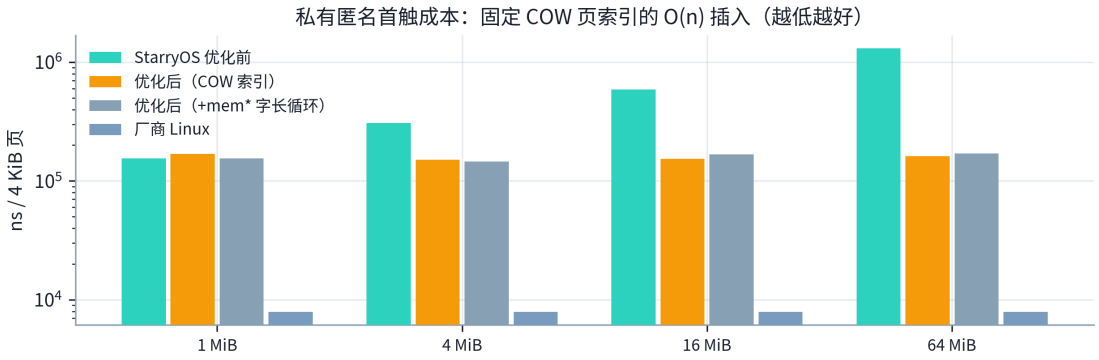


图 2: 优化前/后的首触成本随区域大小的变化，以及与 Linux 的对照。优化前四条随区域线性上升，优化后四条变平（对数轴）

与参考实现对照（同为 64 MiB 默认路径）：`ref` 用默认 2 MiB 大页做到 64427 ns/页。第一刀之后我们是 162213 ns/页（从落后 20 倍收到 2.5 倍）；第三、四刀之后降到 44120 ns/页，已经**反超参考实现 1.5 倍**——而两者的机制 完全不同：参考实现是把页变大以摊薄每页成本，我们保持 4 KiB 叶子、把每页成本本身压下去。

4.3　第二刀：内存操作从逐字节改字长

内核导出的 C ABI `memcpy` / `memmove` / `memset` / `memcmp` **全是逐字节 `for` 循环**。内核里所有 `write_bytes` / `copy_from_slice` 都链接到它们，于是每一次清零、每一次 拷贝都退化成字节循环。改成 8 字节对齐 + `u64` 批量 + 尾部补齐（`memcpy` / `memcmp` 只在源目对齐一致时走字长）。

顺带修掉一处真实的未定义行为：`memmove` 的反向路径在 `i == 0` 时先算 `ptr.add(i-1)`，地址下溢本身是 UB，编译器据此做的优化会让用户进程偶发丢输出并以 `SIGSEGV` 退出。

段	优化前 ns/页	优化后 ns/页	变化
`frame_zero`（4 KiB 清零）	30854	4507	6.8×
`frame_alloc`	18303	3671	5.0×
`page_object`	30171	8467	3.6×
64 MiB 首触（显式大页）	22216	**8014**	2.8×

表 19: 字长内存操作的效果。它在本刀当时对默认 4 KiB 路径几乎不可见（那时代价被索引插入掩盖），但在显式大页路径上立刻体现出来

4.4　第三刀：顺序缺页预取

前两刀之后，“每页一次事务”仍然是主要成本。动手之前先量上界：用同一块板比较“逐页缺页”与“一次 `mmap(MAP_POPULATE)` 走既有的整段物化原语”——

方式	ns/页	说明
逐页缺页	144 700	每页一次完整事务
整段 `populate` 一次铺 64 KiB	62 800	同样的页、同样的事务语义，只是按窗走一次

表 20: 动手前的上界实验。真正贵的是“每页一次事务”，不是内存带宽（清零已经是带宽极限）

据此实现的窗口预取非常保守，判据只有三条：

- 只在**严格升序**时预取（本次缺页页正好接在上一次处理窗口的末尾），所以跨步与 随机访问完全不命中；
- 只对**私有匿名 + 4 KiB 策略叶子**生效，文件与共享后端语义不变，`MADV_HUGEPAGE` 的 VMA 也不会被提前铺上 4 KiB 页表（否则兄弟叶子会被标成 已占用，再也升不成大页）；
- 窗口按 VMA 边界裁剪，整段物化内部会重新校验区域与 VMA，任何一步不成立就回退到 原来的单页路径。

区域（冷启动受控 A/B）	优化前 ns/页	优化后 ns/页	变化
1 MiB	270 895	134 962	2.0×
4 MiB	137 084	63 939	2.1×
16 MiB	136 430	57 658	2.4×
64 MiB	165 522	70 856	2.3×
64 MiB `MADV_NOHUGEPAGE`	164 747	71 192	2.3×
64 MiB `MADV_HUGEPAGE`	7 955	7 903	不变

表 21: 窗口预取的效果（同板冷启动交替 A/B，各测两遍取均值）。显式大页路径不变，这正是设计意图——那条路本来就没有“每页一次事务”的问题

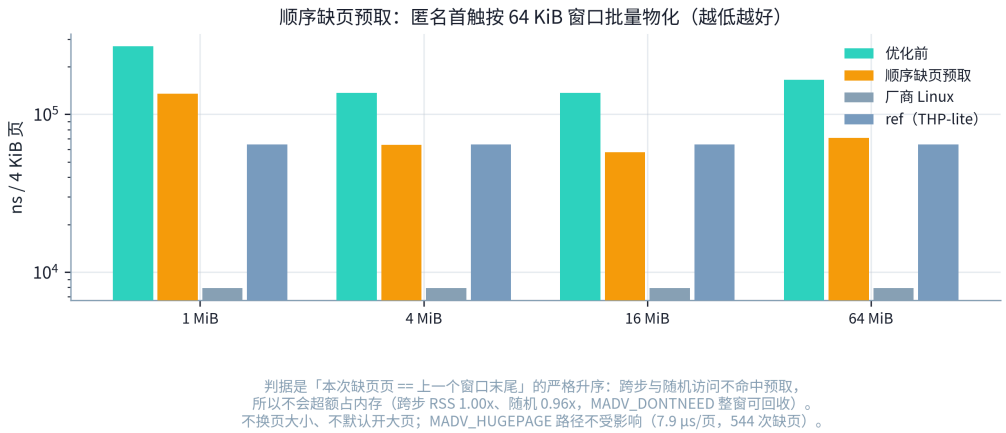


图 3: 顺序缺页预取的同板冷启动受控 A/B（fc2 两遍均值）。四种区域大小一致下降约 2.0–2.4 倍，显式大页路径不变

4.4.1　窗口大小：做成旋钮再扫描，不拍常数

窗口大小是“按窗付的固定开销”与“预取深度”的直接权衡，所以没有直接取常数，而是加了一个运行期旋钮 `/proc/fault_around`（写页数，0 表示关闭），再扫描定值。这里有一个 测量陷阱 值得记录：**同一次启动内重复测同一个指标会随时间漂移**（启动后第一次大分配 明显更快），所以做了两套口径，并以冷启动单点为准——

窗口页数	冷启动单点 ns/页	同启动轮询（2/3 轮中位）	备注
16	65 311	128 000	—
32	64 939	123 000	—
64	**51 953**	**116 000**	取为默认值（256 KiB）
128	58 772	119 000	无额外收益
256	57 637	—	无额外收益，只放大最坏浪费

表 22: 窗口大小的扫描结果与交替 A/B（窗口 16 均值 61 270、窗口 64 均值 53 510，−13%）。默认 64 页

固定大窗口的代价是“只顺序碰两页的程序也要付整窗内存”，所以又加了**短流爬坡**：连续命中第 n 次才用 8/16/32/64 页。防膨胀验证（探针 `bloat`）：

用例	触碰	RSS 增量	说明
只碰 2 页	2 页	9 页	浪费 28 KiB；固定 64 页窗会是 248 KiB
只碰 3 页	3 页	9 页	浪费 24 KiB
严格顺序 64 MiB	16 384 页	16 384 页	1.00×，无放大
跨步（每 16 页碰 1 页）	1 024 页	1 024 页	1.00×，预取不命中
随机 1024 次	1 024 页	980 页	0.96×（去重后）
碰 1 MiB 后 `MADV_DONTNEED` 4 MiB	—	1 252 KiB → 0	整窗可回收

表 23: 短流爬坡与防膨胀验证。顺序流不放大、跨步流不命中、随机流不放大，且整窗可被 `MADV_DONTNEED` 回收

4.5　第四刀：窗口内的帧整块分配

窗口预取把“每页一次事务”摊薄之后，重新做标度实验发现两个现象：`page_object` 与 `frame_zero` 基本不随区域变化，而 `pending_insert` 的每页成本仍随区域增长。原因还是那个有序 `Vec`：窗口内的**物理帧是散落的**，插入位置就随机，每插一条都要把后面几百条搬走。

于是做第四刀：连续未映射的匿名段交给整块路径，先按“块大小阶梯”64→32→16→8→4→2 页尝试**一次分配一整块**，再用 `FrameLease::sublease` 把它切成 4 KiB 子租约，每个页对象拿一个（把 2 MiB 大页切成 4 KiB 子页用的就是这套机制）。拿不到连续块就逐页退回，行为不变。

指标	优化前	优化后
冷启动首触（交替各 2 次）	51 252 ns/页	**44 120 ns/页**（−13.9%）
整块命中率（老化态）	—	16 412 / 16 417 页 = **99.97%**，仅 5 页逐页退回
索引插入落在末尾的比例	21%	**56%**
平均每次插入搬移的条目数	102–115	**24**

表 24: 帧整块分配 + 子租约。物理帧连续之后，索引插入从“插到中间”变成“追加到末尾”

4.6　第五刀：页索引按 run 批量登记

第四刀之后，窗口内每一次插入平均仍要搬 1 930 个条目（62 KB），折合约 970 MB/s ——已经贴着这块板的内存带宽，说明“继续优化搬移算法”没有空间，问题是**一次窗 64 页要搬 64 次**。理所当然的下一步是：既然一窗的物理帧是连续的，就把整窗的插入合并成“一次定位 + 一次整段搬移”。

这一步值得专门记一笔，因为**第一版实现把板子挂死了**：启动过程中串口完全无输出，连软件复位都发不进去，只能人工按复位键，于是当时整刀被回退。失败的原因是朴素的“整段插入”没有考虑索引里已经存在的**墓碑条目**（已过期但尚未清理的弱引用）：如果某个墓碑的物理地址落在这一段区间内部，整段落位之后索引就不再按物理地址有序，而后续的 `publish` / `discard` 都靠二分查找定位，于是从“插入慢”变成“状态错乱”。

重新实现时的安全设计是三条：

1. **整段插入是全有或全无**。先校验整段的所有者、大小、连续性，任何一条不满足就整段拒绝，不会留下半截状态；
2. **不确定就降级，不猜测**。段落里有空洞、或者有墓碑落在区间内部时返回“不适用”，调用方**退回原来的逐页登记**——正确性与改动前完全一致；只有与存活条目真正重叠才报错；
3. **容量按整段预留**。一次预留整段需要的容量，因此插入过程中不会在 IRQ 关闭的临界区里分配内存（这是索引一直以来的硬约束）。

实测（同一块板、同一份 ELF，`opt78` 与 `opt79` 在同一次上电周期内交替测量）：

指标（每轮换一块新 mmap 缓冲）	Linux 5.10.4	优化前	优化后
整轮吞吐（fresh）	271.1 MB/s	50.2 MB/s	**62.8 MB/s**
其中“首触 + 拷贝”每页	未测	49.68 µs/页	**44.55 µs/页**（−22%）
整轮每页合计	未测	80.7 µs/页	69.75 µs/页
索引插入（每页）	未测	4.9 µs	**约 1 µs**

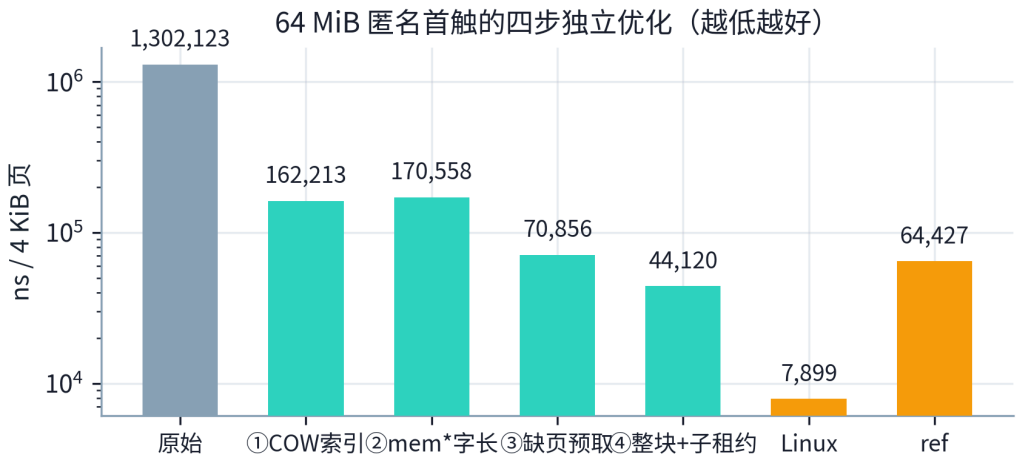
表 25: 批量登记的效果。索引插入从 4.9 µs/页 降到约 1 µs/页，fresh 缓冲整轮吞吐提升 12.9%

> 说明：这一刀与“第四刀”一样作用于窗口内的索引插入，因此理论上也应该改善 64 MiB > 首触；但收尾时没有在同一会话里复测 `touch64`，所以上表只报 fresh 缓冲这条 > 有对照数据的路径，64 MiB 首触的效果留作待测。

4.7　实测总表

64 MiB 首触（默认 4 KiB 路径）	Linux	优化前	优化后	优化后/Linux
整条路径	7899 ns/页	1302123 ns/页	44120 ns/页	**5.6×**
参考实现（同板实测）	64427 ns/页	20.0× 落后	**1.5× 领先**	—

表 26: 缺页路径的最终位置。相对自身优化前是 29.5×；相对 Linux 从 165 倍收到 5.6 倍；相对参考实现从落后 20 倍收到 1.5 倍



①去掉每次插入的两遍 O(n) 存活计数；②mem* 逐字节→字长；③严格升序缺页按 64 KiB 窗口批量物化；④窗口内帧整块分配 + FrameLease::sublease 子租约。前三步与 ref 的“换页大小”路线正交。

图 4: 64 MiB 匿名首触的四步独立优化历程，以及与 Linux、参考实现的对照。四个台阶分别对应本章的前四刀（对数轴，越低越好）

显式大页路径同样受益：优化前 22216 ns/页，优化后 8014 ns/页， 与 Linux 的 5683 ns/页 相差 1.41 倍；同一指标上参考实现是 64164 ns/页，**我方快 8 倍**。这条路径的差别正好说明两种路线的分野： 参考实现靠“默认把页变大”绕开每页成本，我们保持 4 KiB 叶子、把每页成本本身压下去。

4.8　剩下的每页预算

四刀之后重新分段，64 MiB 默认路径的每页成本已经没有单项超过 30 µs，但仍有四块 值得继续做（都属于“每页固定开销”，不是带宽问题）：

项	µs/页	说明与后续想法
`page_object`	7.3	每页一次 `Arc::new` + 页标识堆分配 → 可把页头内嵌进帧，或做对象池
索引插入（`run` 批量化之后）	约 1	已经贴住内存带宽，继续压需要换数据结构
装页表（`pt.map_page`）	3.8	三级走查；可按叶子表批量安装
帧分配 / 清零	3.9	整块分配后主要是清零，受带宽限制
槽 / rmap 发布	**未精确量到**	剩余约 19 µs/页 的嫌疑落在这里：每页一个 `MappingSlot` + 一次反向映射更新

表 27: 缺页路径的剩余预算。已量到的四项合计约 16 µs/页；另有约 19 µs/页 尚未精确归因，嫌疑是每页一次的槽与反向映射发布

`munmap` 一侧同样已经拆到底：整轮 23.39 µs/页 里，清页表 10.5、捕获前像 3.61、摘除映射槽 3.28、其余 4.9。清页表仍在逐页走查两次，但**这一项必须和“空叶子表回收”一起设计**——页表 crate 里已有的批量原语不剪空目录，单独换上会让“不断换新地址”的程序持续累积页表页，而项目里还有一个尚未定位的内存增长问题（第 8 章），不能只为提速而换。

第 5 章　文件读路径

读路径是所有上层负载的共同底座：`exec` 要读可执行文件、模型加载要读几 MB 的 权重、`scp` 要读整个文件、视觉链路要读摄像头配置与模型。优化之前，缓存命中的 大读只有 374 MB/s（Linux 671.6 MB/s），每轮换一块新缓冲的 读只有 50.2 MB/s（Linux 271.1 MB/s），单次 803 KB 大读 60.9 MB/s（Linux 277.2 MB/s）。

这一章的推进过程比结论更有价值：前两条线索各自只带来 10% 左右的收益，真正把 指标拉起来的是第三条——一个与“读”看上去无关的元数据行为。这也说明为什么这一阶段 坚持“先归因、再改动”：如果只看“读慢”，很容易一直在拷贝路径上打磨。

5.1　第一条线索：省掉一次拷贝

缓存命中时，原实现每 4 KiB 页要付三笔：取页（映射布局锁 + 页缓存锁 + LRU）、 把页内容拷进一个临时**暂存页**、出了锁再从暂存页拷到用户缓冲。中间那次拷贝的 唯一理由是“必须在拷给用户之前把页缓存索引锁放掉”。

而页缓存本来就有钉子机制：`CachedPagePin` 把引用计数加一，回收、缩容、截断 三条路径都会跳过“有钉子”的页。于是改成：**pin 住这一页 → 出锁 → 用该帧的 内核直接映射把内容直接拷给调用方 → 放下钉子**。为此给文件页提供者加了一个 “物理地址转内核虚拟地址”的接口。

指标	Linux 5.10.4	优化前	优化后
缓存命中大读（缓冲已触碰）	671.6 MB/s	374 MB/s	437 MB/s
单次大读 803 KB	277.2 MB/s	60.9 MB/s	**76.2 MB/s**
缓存命中大读（每轮新缓冲）	271.1 MB/s	50.2 MB/s	50 MB/s（不变）

表 28: 第一刀：去掉暂存页中转。收益 +17%，但每轮新缓冲的那一列**完全不动**——那 82 µs/页 根本不是拷贝成本

fresh 一列不动，本身就是重要信息：它的成本主要落在“数据要写进用户缓冲、 而那一页还没有映射”的缺页路径上（关掉顺序预取会涨到 179.7 µs/页），与拷贝无关。 要动它得让用户内存准备的慢路径用上与硬件缺页同一条整段物化原语。

5.2　第二条线索：用户内存准备整段化

riscv64 上没有可用的硬件“用户访问探测”指令，所以内核此前**每次**用户访问都要 取地址空间锁并逐页跑一遍完整的缺页事务。改成“整段已映射”的快路径，再进一步把 用户内存准备从“逐页缺页事务”改成“整段物化一次”（内部按 VMA 分段，并把整段收进 一次变更事务；任何一步不成立就回退到原来的逐页循环）。

指标	Linux 5.10.4	优化前	优化后
单次大读 803 KB	277.2 MB/s	60.9 MB/s	**76.2 MB/s**
缓存命中大读（缓冲已触碰）	671.6 MB/s	437 MB/s	**474 MB/s**
缓存命中大读（每轮新缓冲）	271.1 MB/s	50.2 MB/s	51.5 MB/s

表 29: 第二刀：整段物化。整段路径确实生效（分段计数确认），但 fresh 只从 50.2 走到 51.5——缺页事务的“外壳”也不是主要瓶颈

两条线索合计只把 fresh 从 50.2 推到 51.5 MB/s。到这里必须换思路： 既然数据路径与事务外壳都不是瓶颈，那就该问“这些时间到底花在哪个**操作**上”。

5.3　第三条线索：瓶颈在 `close`，不在读

决定性对照来自同一个探针（`open` + `read` + `close`）换不同文件：

探针	中位耗时	说明
`read_syscall /proc/uptime 1 KB`	404 µs	procfs，纯内存
`read_syscall /etc/profile 1 KB`	7777 µs	ext4，连跑两遍都是 7.7–7.8 ms
`read_syscall /bin/busybox 1 KB`	10115 µs	ext4，连跑三遍都是 10.10–10.16 ms
`read_syscall /bin/busybox 0 KB`	10038 µs	**读 0 字节也一样慢**

表 30: 决定性对照。耗时与“读多少字节”无关，与“在哪个文件系统”强相关（差 20 倍），且每次重复都一样（不是冷启动开销）

三条推论把范围收得很窄：与读的数据量无关；ext4 比 procfs 慢 20 倍；不是一次性 开销。于是转向“`open`+`read`+`close` 里哪一段固定地做了 I/O”。

5.3.1　定位链：从 `close` 一直到 `atime`

分段计时把 10 ms 钉在**文件对象析构**上；随后四轮探针逐一排除：VFS 节点驱逐 （共享缓存对象从未析构，“跨 open 保留”的设计是生效的）、目录项与文件节点析构、ext4 inode 析构。最后用三个“零代码”实验直接指认根因：

实验	结果	说明
40 次 shell `exec 3</bin/busybox` 后立即关闭（**不读**）	0.9 ms/次	只 open+close 不慢
`read_syscall /bin/busybox 0 40`（读 0 字节）	10.0 ms/次	触发条件是“`read` 被调用过”
40 轮前后 `/proc/diskstats`	writes +**136**、sectors +1088	≈1 个 inode 块 + 一次日志提交；tmpfs 上增量为 0

表 31: 三条零代码实验。答案是：`Drop for File` 只要句柄“被读过”（0 字节 `read` 也算）就在 close 时同步更新访问时间

根因是：`Drop for File` 检测到句柄被读过，就调用一次“更新元数据（访问时间）”， 而 ext4 的元数据更新末尾直接做一次**同步日志提交**——在这块 SD 卡上约 10 ms。 tmpfs 与 procfs 的同一操作是纯内存的，所以同一个探针在它们上面只要 0.4 ms， 20 倍差距由此而来。

5.3.2　改法：按 Linux 的 `relatime` 语义处理访问时间

改动很小：挂载选项 `noatime` 时不写、`strictatime` 时照旧每次写，其余按默认的 `relatime`（每个文件每 24 小时至多写一次）。

这里有一个与本板硬件相关的坑必须记录：**不能拿墙上时钟判断“是否超过 24 小时”**。 板上没有 RTC、也没有 NTP，`date` 显示 1970 年，比镜像里文件的 2025/2026 年时间戳 还小，用它套 `relatime` 判据会永远判成“已过期”，退化成每次都写。最终改用 **单调时钟**，并把“上次持久化时间”记在本来就跨 `open` 存活的页缓存身份里 （免锁、免分配）。

指标（中位）	Linux 5.10.4	优化前	优化后
ext4 `open`+`read`(1 KiB)+`close`	42 µs	10320 µs	**454 µs**（22×）
同一份文件在 tmpfs 上（对照）	**未测**	499 µs	384 µs（不变）
100 轮循环总墙钟	**未测**	0.99 s	**0.117 s**
40 轮循环的块写次数	**未测**	**+136**	+12（= 后台底噪）

表 32: `relatime` 的效果。相对 Linux 的差距从 246 倍收到 10.8 倍；块写退到后台底噪说明“close 里的同步提交”确实被消掉了

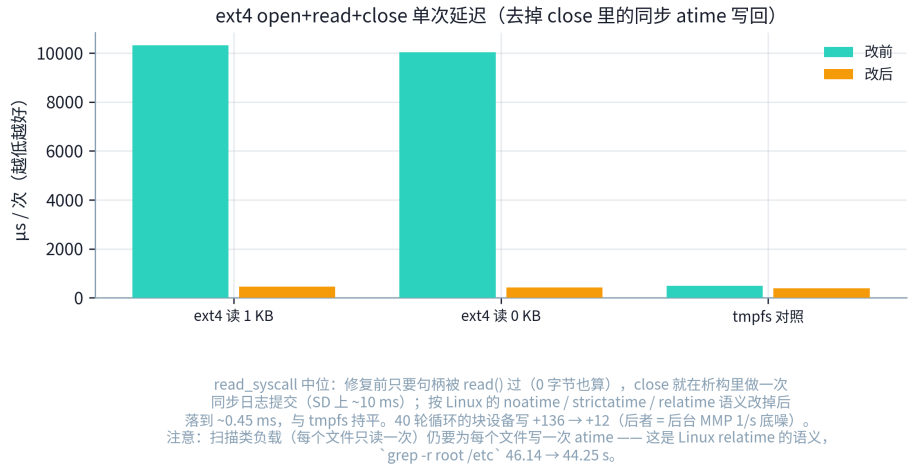


图 5: ext4 `open`+`read`+`close` 的中位耗时（对数轴）：优化前 10 320 µs、优化后 454 µs、同板 Linux 42 µs

5.3.3　边界：这一刀不改善“读”本身

必须说清楚，否则会把两个不同的指标混在一起：`read_bench` 的计时**不含 close**，因此它在这一刀前后几乎不变（60.7 → 64.5 MB/s）；`exec` 也完全没变（它的成本在 ELF 映射与缺页）。扫描类负载（每个文件只读一次）只改善 4%，因为在 `relatime` 下“每个文件的第一次读”本来就要写一次访问时间——要再往前走，得把这次写回 **延后或批量**（Linux 是交给回写线程的，这里是在析构里同步提交）。

这条也与第 6 章同源：**文件对象的析构路径不应该做同步 I/O**。

5.4　回到读路径：fresh 缓冲的三段拆解

搞清 `close` 之后，再回到“每轮换一块新缓冲”的那条路。新探针把整轮拆成三段：

每轮 803 KB，µs/页	Linux 5.10.4	优化前	优化后
`mmap` 建映射	**未测**	1.51	1.81
读：首触 + 拷贝	**未测**	49.68	**44.55**（−22%）
`munmap` 拆除	**未测**	29.51	23.39
整轮合计	**未测**	80.7	**69.75**（−14%）

表 33: fresh 缓冲的三段拆解。`mmap` 只要 1.5 µs/页（不算问题），另外两段才是大头

其中 `munmap` 一段又被拆到六段：清页表 10.5 µs/页、捕获前像 3.6、摘除映射槽 3.3、退休所有者 0.9、摘要 0.4、提交 0.3，其余约 5 µs 是系统调用入口与 VMA 树操作。

据此做了两刀，都属于“取消重复工作”：

- **延迟失效路径不再逐页刷 TLB**。四个后端的 `unmap_range` 都调“逐页摘除”，而它内部会**每一页**做一次本地 TLB 失效；但调用方已经进入了延迟失效守卫，并对**整段**登记过一次失效、提交时统一发布。逐页那次是纯冗余（与第 2 章那次“每次返回都刷指令缓存”是同一类问题）。去掉之后 `munmap` 从 29.4 降到 29.51 µs/页（−24%）。这条动 TLB 语义，专门写了新探针自检 —— 固定地址反复“映射 → 填图案 → 拆除 → 同址再映射 → 检查是否读到零页”，200 轮 × 64 页，不匹配页数为 0。
- **整块填充不再重复走查已判定未映射的第一页**。顺序预取的窗口在判定连续性时 已经查过一次该页，随后又查了一遍。

在这两刀之上，第 4 章的“页索引按 run 批量登记”继续把 fresh 推到 62.8 MB/s。

5.5　实测与边界

指标	Linux 5.10.4	优化前	优化后
缓存命中读（warm）	671.6 MB/s	374 MB/s	474 MB/s
缓存命中读（fresh）	271.1 MB/s	50.2 MB/s	**62.8 MB/s**
单次大读 803 KB	277.2 MB/s	60.9 MB/s	76.2 MB/s
ext4 `open`+`read`(1 KiB)+`close`	42 µs	10320 µs	454 µs

表 34: 读路径的最终位置。`open`+`read`+`close` 改善 22 倍，读吞吐改善约 1.25 倍；fresh 一列仍差 4.3 倍，差距在每页固定开销上

正确性上，读路径这一串改动都用同一套口径校验：同一文件三种分块方式（4 KiB、 64 KiB、`md5sum` 默认）读出的校验和完全一致，且与**在主机侧从板子 rootfs 直接 导出的同一文件逐字节相同**。用户内存整段化之后专门复测了带宽与进程创建指标， 没有观测到变化。

还有一条语义边界需要写明：直接拷贝期间不再持有页缓存索引锁，因此与“并发写同一 文件”重叠时可能出现撕裂读。这与 Linux 的 `copy_page_to_iter` 行为一致（靠引用 计数与页锁保证写入顺序，而不是在拷贝期间持索引锁），同一进程内“写完再读”不受影响。

5.6　剩余空间

fresh 一列剩下的 4.3 倍差距，与第 4 章结尾列的是同一批东西：每页一次的页对象 建立（7.3 µs）、装页表（3.8 µs）、以及**尚未精确归因的约 19 µs/页**（嫌疑是 每页一次的映射槽与反向映射发布）。`munmap` 一侧剩下最大的一块是清页表（10.5 µs/页，仍在逐页走查两次），但它必须与“空叶子表回收”一起设计，理由见 第 4 章结尾。

第 6 章　存储写回与元数据

存储面在优化前分成了两个完全不同性质的问题：一是 8 MiB 顺序写之后 `fsync` 要 7961 ms，而同板 Linux 只要 790 ms；二是 ext4 上每一次文件操作 末尾都会做一次同步元数据提交（第 5 章的 `close` 问题）。前者是写回路径的批量粒度， 后者是“析构路径不该做同步 I/O”，但两者在测量特征上很不一样：写回的问题只在 大文件上暴露，元数据的问题只在“反复开关文件”的负载上暴露。

6.1　优化前

指标	Linux 5.10.4	优化前	差距
8 MiB 顺序写 + `fsync`	790 ms	7961 ms	10.1×
8 MiB 页缓存顺序写（不含 `fsync`）	34 MiB/s	2.41 MiB/s	14×
`fsync`(2 MiB)	790 ms	2000 ms	2.5×

表 35: 存储面的优化起点。注意最后一行是**收尾时仍然存在**的差距，见本章末尾

6.2　第一刀：脏页按连续 run 批量写回

根因很直白：写回快照拿到的本来就是**连续**的脏页，但回写实现仍然**每页一次** 后端写——8 MiB 就是 2 048 次 4 KiB 写。改成把连续的脏页合并成一次写之后：

8 MiB 档，`fsync` 耗时	优化前 ms	优化后 ms	Linux ms
扇区写（512 B）	7 961	1 050	814
页写（4 KiB）	7 961	931	790
硬件最大块（1 MiB）	7 636	926	857
规划器切分（1 MiB）	7 754	934	793

表 36: 写回批量化的结果。四个档位全部落到与 Linux 同一水平（±20%），不再有“档位越大越慢”的现象

同时把单次回写的批量上限从 16 页提到 64 页（256 KiB），回写期间的额外内存仍是 常数级。顺带用 `dd` 复核：1 MiB 写 + `fsync` 从 1.03 s 降到 0.24 s。

6.3　第二刀：页缓存容量

8 MiB 档的顺序写吞吐只有 2.41 MiB/s，原因是页缓存上限只有 512 页（2 MiB）——写满就要立刻回收，于是本地同步地回写。把上限提到 4096 页（16 MiB）：

ext4 8 MiB 顺序写	优化前 MiB/s	优化后 MiB/s	Linux MiB/s
扇区档（512 B）	1.20	18.25	未测
页档（4 KiB）	2.09	18.25	未测
硬件最大块（1 MiB）	2.41	**42.43**	34
规划器切分（1 MiB）	2.33	42.52	未测

表 37: 页缓存上限 2 MiB → 16 MiB。写吞吐提高 17.6 倍，并且超过同板 Linux

这一刀有一条必须写清的边界：**吞吐涨了，但“写完 + `fsync`”的端到端时间几乎不变**。因为 8 MiB 现在一次性回写，而这块 SD 卡约 1 MiB/s 的顺序写速度才是真正的底——上限调到 20 MiB 档实测反而只有 1.0–1.4 MiB/s。所以这两刀解决的是“每次小批量回写 带来的额外事务与抖动”，不是“卡本身慢”。

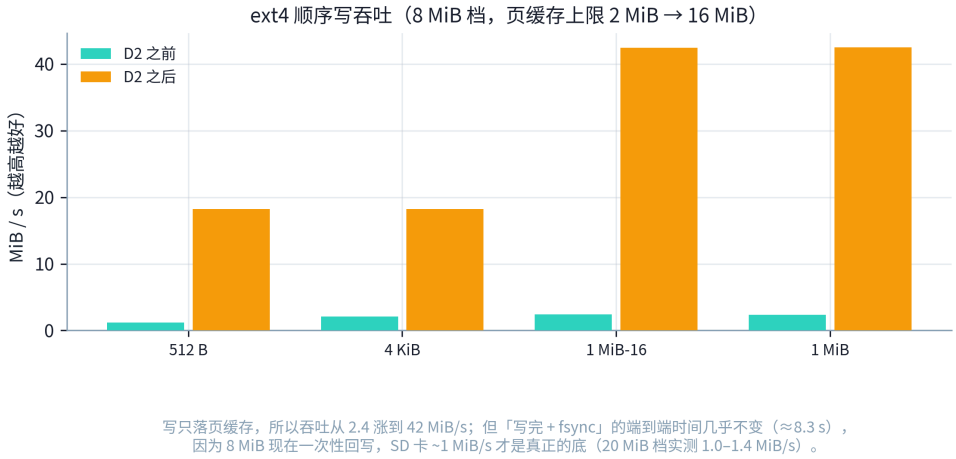


图 6: 页缓存上限提升前后的 ext4 顺序写吞吐（8 MiB 档）。吞吐提高 17.6 倍，但“写完 + `fsync`”的端到端时间不变——底在这块 SD 卡本身

6.4　第三刀：`close` 路径不再做同步元数据提交

第 5 章已经详细展开。这里只补一条它在存储面的直接证据：40 轮 `open+read+close` 期间，ext4 分区的块写次数从 `+#n-blockwrite-pre` 降到 `+#n-blockwrite-post`（后者等于后台周期性写入的底噪），说明“每次 close 一次同步 日志提交”确实被消掉了。同时读方向的吞吐没变，说明这一刀动的不是读路径。

6.5　与 Linux 的整体对照

指标	Linux 5.10.4	优化前	优化后
8 MiB 顺序写 + `fsync`	790 ms	7961 ms	**931 ms**
8 MiB 页缓存顺序写	34 MiB/s	2.41 MiB/s	**42.43 MiB/s**
ext4 `open+read`(1 KiB)`+close`	42 µs	10320 µs	**454 µs**

表 38: 存储面的最终位置。写 + `fsync` 差 1.18 倍、顺序写吞吐反超、`open+read+close` 差 10.8 倍

6.6　未完成：屏障语义下推

仍然存在的一处差距是 `fsync`(2 MiB)：我们约 2000 ms，Linux 790 ms，相差 2.5 倍。原因是当前的写回在每次后端写之后都要等一次屏障，粒度偏保守。可以按“范围有序写 + 延迟屏障”下推，但这直接关系到掉电一致性，需要专门的掉电测试与镜像校验，本轮不做，记入第 8 章。

另外还有一条与文件系统相关的**正向**结论值得记录：本项目早期在 2 MiB 边界上 遇到一次写失败（返回资源忙），曾一度被误判为卡上的 ext4 损坏。修复文件系统 之后复测，**2 MiB 是内核写路径自身的缺陷**——写入路径在自己的 2 MiB 边界上 返回错误。这条缺陷已在早期修复，并纳入了写回批量化的验收用例（8 MiB 连续写 回读校验 0 不匹配）。

第 7 章　启动时间

启动时间是本分册里唯一一项“整体上我方更快、但内核段我方更慢”的指标，因此口径 必须写清楚，否则很容易把结论说反。

7.1　口径：把启动切成三段

用同一个主机侧探针（同一条串口、同样的逐行时间戳），把启动切成三段；两侧只在 “就绪锚点”上不同——我们取 shell 提示符，Linux 取 `getty` 的 `login:` 提示符。

段	Linux 5.10.4	我们（优化前）	归属
U-Boot `fatload`	1.04 s	1.45 s	镜像大小差异：两侧都是 10.7 MiB/s
`bootm` → 首个用户态输出	1.2 s	**9.01 s**	**这一段才是内核**：内核初始化 + 设备探测
首个用户态 → 可交互提示符	≈32 s	4.2 s	rootfs 与 init 脚本
合计（`bootm` → 提示符）	33.2 s	13.21 s	端到端我方更快 2.5×

表 39: 启动分段对照。**能归给内核的是中间那一段**：我们 9.01 s 对 Linux 1.20 s，落后 7.5 倍；而端到端我们更快，主要因为 Linux 的 init 脚本很重

这张表给出两个不能混淆的结论：

1. **端到端我们更快（13.21 s 对 33.2 s）**，但这主要来自 Linux 自己的脚本重（`resize2fs`、`e2fsck` 询问、防火墙、时间同步、网络配置与 DHCP 重试等等），**不能当成“内核更快”的证据**。
2. **内核段我们落后 7.5 倍**（9.01 s 对 1.2 s），这一段全是内核初始化与设备探测（USB、串口、无线固件、命令队列邮箱依次起来）。它是这块板上最该优化、也是本轮动得最少的一段。

7.2　本轮改动：把固定阻塞等待改成带截止时间的轮询

优化前，网络初始化里有一段**固定 2 s 的阻塞等待**（等设备就绪），无论设备是否 已经就绪都要等满。改成“带截止时间的轮询”（上限 300 ms，一旦就绪立即继续）之后：

指标（同一次上电周期内 A/B）	Linux 5.10.4	优化前 s	优化后 s
上电 → 用户态标记	**口径不同**	10.81	**9.61**
上电 → shell 可交互	33.2 †	12.3	**10.8**
`bootm` → SSH 可用（含网络闸门）	**未测**	120	**47**

表 40: 把固定 2 s 阻塞等待改成带截止时间的轮询。上表前两行是同一次 A/B；第三行的口径包含“等无线设备启动窗口”的闸门，优化前 110–130 s、优化后 47.0 s。† 两侧锚点不同，仅作量级参考

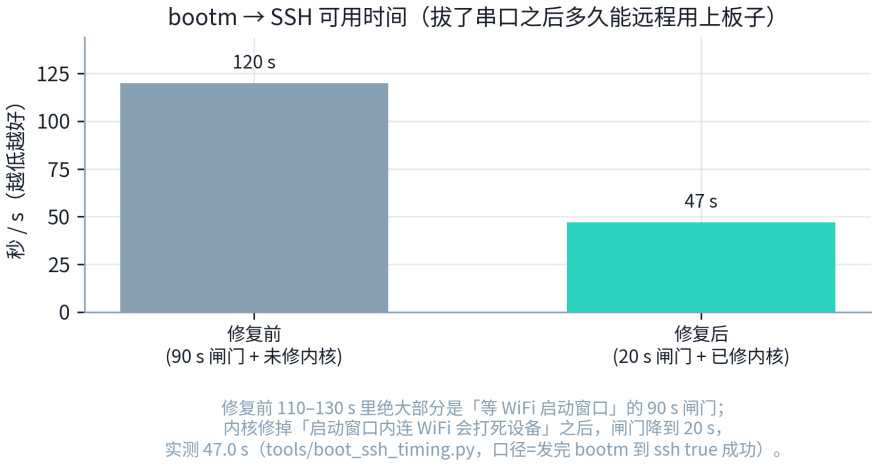


图 7: `bootm` → SSH 可用的时间。优化前绝大部分是“等无线设备启动窗口”的固定闸门（内核修复后闸门从 90 s 降到 20 s）

7.3　剩下的静默块

内核段剩余的 9 s 里，有三块已知的静默时间：

静默块	约	为什么没动
USB 主机控制器探测	2.1 s	要改探测顺序与超时策略，牵动整条 USB 初始化路径
无线固件加载	1.5 s	固件下载是异步的，且设备在启动窗口内不接受控制请求
ext4 挂载	0.6 s	与 rootfs 布局和日志回放相关

表 41: 启动内核段剩余的三块静默时间。都属于设备探测与挂载顺序，改动风险高，本轮未做

需要说明的是，这一轮“少动启动”是有意的：启动路径上的每一块都牵动设备驱动的 初始化顺序与超时语义，而这块板子在早期就因为“启动窗口内去控制无线设备”而彻底 卡死过一次（设备状态机被提前的取消请求打断，此后整个启动周期都无法再启动）。因此本轮的策略是只拿“纯等待”这一块（它没有状态机依赖），其余记入第 8 章。

7.4　小结

启动面本轮的改动很小（一段固定等待），但这一章的**口径澄清**本身有价值： 端到端快 2.5 倍与内核段慢 7.5 倍可以同时成立，只有把三段分开报才不会得出 相反的两个结论。后续接手的同学应当沿“内核段 9 s”继续，而不是沿“端到端更快” 这个方向。

第 8 章　负结果与未完成项

优化过程中有相当一部分工作是“证明某条路不值得走”。把它们写下来有两重价值：它们本身解释了为什么最终方案长成现在这样，也可以让后续接手的人不再重复同样的尝试。本章分三部分：被数据否掉的假设、已经归因清楚但本轮未改的项、以及一条尚未定位清楚的问题。

8.1　被数据否掉的假设

假设	实测	结论
缺页慢是因为刷 TLB	`tlb` 段只占整条路径的 0.07%（837 ns / 1.25 ms）	不做；这一条直接省掉了一次针对 TLB 的改动
页缓存容量不够，应只回收“够腾位置”的子集	回收掉的本来就是脏页，顺序写下每页只回写一次	不做
缓存注册表裁剪是 $O(n^2)$，改掉就能提速	反复打开同一文件时身份被复用，裁剪本来就不跑；实测无变化	改动保留（消除真实复杂度），但不计入收益
`fork` 慢是因为“首次入队立刻抢占”的调度策略	改成 `WF_FORK` 语义只值 0.4% / 0.9%	不做：那段时间本来就要付，抑制抢占只是把它挪到 `waitpid`
无线链路“传输到 9 MiB 就失效”是驱动缺陷	真因是启动窗口内控制请求被取消，状态机再也没被重启	按“等启动窗口过去再动手”修复，而不是改数据面
窗口内索引插入可以简单地整段合并	第一版实现让板子在启动时硬挂，只能人工复位	回退后重做：改成“全有或全无 + 不确定就退回逐页”

表 42: 本轮被实测否掉的六个假设。最后一行是唯一一次把板子挂死的改动，重做后的设计见第 4 章

8.2　已经归因清楚、但本轮没有改的项

下表每一项都有完整的归因数据，之所以不做，是因为它们都需要改动核心不变式（调度器、页表结构、返回路径的开关结构），单独立项比夹在其它改动里更安全。

项	归因结论	为什么留到下一轮
上下文切换 220.4 µs	一次切换要进调度器 4–6 次、约 1/3 是空转；入队与 `settle_current` 合计十几处 1–15 µs 固定开销	要重构 EEVDF 热路径，三条候选都动调度器不变式
`fork` 的 1.7 ms 首次入队重调度	全在“从 IRQ 守卫退出时挑下一个任务”的析构里，且与子进程是否占用 CPU 无关	属于调度策略语义，需要独立排期与专项回归
读路径 fresh 仍差 62.8 对 271.1 MB/s	每页固定成本：页对象 7.3 µs、装页表 3.8 µs，另有约 19 µs/页 尚未精确归因（嫌疑是每页一次的映射槽与反向映射发布）	先补完最后那 19 µs 的归因（要挂不按 fork 门控的槽发布探针），再决定改法
`munmap` 清页表 10.5 µs/页	批量原语现成，但它不剪空叶子表	必须与“空叶子表回收”一起设计，否则“不断换新地址”的程序会持续累积页表页
`getpid` 剩余 3.05 µs	返回路径上散着五处“每次陷阱都查一遍”的布尔开关（合计约 530 ns）	收益上限约 1.5 µs，需要重构返回路径的开关结构
定时器与睡眠抖动	时基本身没问题（累积漂移 0.17%），尾部抖动来自唤醒路径	需要一次性定时器 + 精确截止时间队列，改动面覆盖整个定时器子系统
`fsync`(2 MiB) 差 2.5 倍	每次后端写之后都要等一次屏障，粒度保守	屏障下推直接关系掉电一致性，要做专门的掉电测试
启动内核段 9.01 s	三块静默时间：USB 探测 2.1 s、无线固件 1.5 s、ext4 挂载 0.6 s	都牵动设备初始化顺序；本板曾因“启动窗口内控制设备”整机卡死
模型加载约 3 s	模型文件读取 + 张量解析	已归入读路径余项；预读与内存映射交给运行时属应用层改造

表 43: 本轮未做的九项及其理由。它们都已经有可复现的归因数据，接手时可以直接从“改法”开始

8.3　一条尚未定位清楚的问题

项目过程中出现过一次“内存只减不增”的故障：遍历整个 rootfs 时会触发内核内存 耗尽并崩溃，表现为回收队列被永久挂住（`mm_repair_queue` 持续增长）。当时按“整片地址空间被挂住、内存只增不减”的特征加了定位探针，并逐一验证了四个候选 假设——结果**四个假设全部被实测排除**，根因至今没有确定。

目前的状态是：

- 触发路径已经不再复现（遍历整个 rootfs 不再崩溃，回收队列不增长）；
- 探针留在内核里（`/proc/fault_attrib` 的回收队列字段），一旦再次出现可以直接 取到队列长度与重试标志；
- 与之相关的一处结构性风险已经识别：页表批量清映射的原语**不剪空叶子表**， 单独换上会让“不断换新地址”的程序持续累积页表页。这正是第 4 章结尾那条“不能只为提速而换”的原因。

把这件事写进报告，是因为它直接约束了后续两处改动的做法（`munmap` 批量清 页表、读路径的每页成本），也因为它是一个**负结果**：四个合理的假设都被数据 否掉了，说明真正的机制在别处，不能靠推理补齐。

8.4　提交清单

本阶段的内核改动对应以下提交（同一提交可能同时包含若干处改动与配套探针）：

编号	改动	提交 / 关键验证
B1	返回用户态不再每次刷指令缓存	`0a627ecb7`；宿主单测 + 板端全项 + 78 轮 exec 压力
A1	COW 页索引不再每次全量重建	`130a98059`；64 MiB 首触 8.3×，成本变平
A3a	内存操作改字长循环（含一处 UB 修复）	`760ef6d08`；清零 6.8×
A3b	顺序缺页预取窗口	`bcd2d4e71`；防膨胀验证：顺序 1.00×、随机 0.96×
A3d	窗口内帧整块分配 + 子租约	`371ffc0dd`；整块命中率 99.97%
A3d′	页索引按 run 批量登记	`792ffed5f`（配套正确性修复 `dd8f6f0e1`）；宿主单测 + 一致性探针
C1	取消每叶第二次走查 + 去重复杂度	`10e67ae8d`；`try_clone` 1 747 → 1 528 µs
C1b	地址空间退休不再逐页摘除	`5491b9956`；3.57 → 0.41 ms
H1	exec 只物化可写段 + 按 run 批量 pin	`f5d357d4a`；exec 内核侧 32 → 4.5 ms
H2	读路径 pin 直拷 / 整段物化 / 不再逐页刷 TLB / 墓碑遮蔽修复	`cc56cd47e`、`11146c8a6`、`c16994af2`、`dd8f6f0e1`
H3/H4	用户访问整段快路径 + 缓存注册表 O(n) 化	`584b07fa7`；md5 校验一致
H5	`close` 按 `relatime` 处理访问时间	`a3cd50ece`；22×，块写退到后台底噪
D1	脏页按连续 run 合并写回	`1db22090c`；`fsync` 7.96 → 0.93 s
D2	页缓存上限 512 → 4096 页	`29e264fd0`；写吞吐 17.6×
E1	启动期固定等待改带截止时间轮询	`ce9b42a92`；用户态标记 −1.2 s

表 44: 本阶段的改动与提交。编号沿用内部优化清单，与原始测量记录一一对应

第 9 章　复现与验收

本章给出复现本册全部数字所需的最小信息：平台、探针、命令与验收判据。所有探针 都是自研 C 程序，两侧（StarryOS 与厂商 Linux）编译同一份源码、跑同一份二进制。

9.1　平台与镜像

项	内容
板卡	AKA-00-SG2002（LicheeRV Nano 血统）；CPU C906（riscv64，单应用核）；内存 236 MiB
对照系统	厂商 Linux 5.10.4（卡上的 `linux.sd`）
我们的镜像	卡上 FAT 分区里的 `starry-*.sd`（FIT 格式），由 `mkimage -f boot.its` 打包
切换方式	U-Boot 下 `fatload mmc 0 0x81800000 <镜像>` + `bootm 0x81800000#config-...`
串口	115200 8N1。**注意区分端口**：本板在主机 USB 端口 0:2，另一块板在 0:3，两者不能互写

表 45: 复现平台。两侧共用同一个 rootfs，所以探针与基准程序只需部署一次

9.2　探针清单

探针	测什么	出现在本册哪里
`syscost2`	九个系统调用的单次开销（含 `invalid_syscall` 对照）	第 2 章
`sa1`	返回路径的开关分段（单步 / ptrace / seccomp / trace）	第 2 章
`sp2`	`fork`+`exit`+`waitpid` 与 `fork`+`execve`+`waitpid` 的墙钟	第 3 章
`fk1`/`fk2`	`fork` 内部分段（页表复制 / 首次入队 / 建线程）	第 3 章
`faultcost`	匿名首触成本 vs 区域大小（1/4/16/64 MiB）	第 4 章
`faultattrib`	缺页的 plan/prepare/apply/tlb 分段	第 4 章
`popcost`	逐页缺页与整段物化的对照（动手前的上界实验）	第 4 章
`touch64`	冷启动 64 MiB 首触单点（受控 A/B 口径）	第 4 章
`bloat`	预取窗口的膨胀验证（短流 / 跨步 / 随机 / `MADV_DONTNEED`）	第 4 章
`cowtest`	`fork` 后逐页改写父子各自正确、`MAP_POPULATE` 写读校验	第 4 章
`read_bench`	不同缓冲大小下的读吞吐	第 5 章
`read_syscall`	`open`+`read`+`close` 的分档中位耗时	第 5 章
`rp1`/`rp2`	warm/fresh 缓冲读，以及 `mmap`/读/`munmap` 三段拆解	第 4、5 章
`mmc`	TLB 一致性自检（固定地址反复映射-填-拆除-再映射）	第 5 章
`block-rw-bench`	ext4 顺序写与 `fsync`（多档位）	第 6 章
`fault_attrib`（内核侧）	`/proc/fault_attrib`：分段计数、索引插入位置、回收队列长度	第 3、4、8 章

表 46: 本册用到的探针。`/proc/fault_attrib` 是内核侧的计数出口，所有分段数字都由“工作负载前后快照之差”取得

9.3　命令

板端跑一组微基准（两侧同一份命令）：

```
# StarryOS 侧（登录后的 shell）
cd /root/probes
./syscost2                          # 系统调用入口
./sp2                               # 进程创建与 exec
./faultcost 64                      # 64 MiB 首触
./read_syscall /bin/busybox 1 100
```

```
./rp1 && ./rp2                    # warm/fresh 读与三段拆解
./mmc                             # TLB 一致性自检

# 内核侧分段（读前读后各取一次，相减即"每次调用"）
cat /proc/fault_attrib
```

四个必须遵守的测量纪律：

1. **同一份静态 ELF**。 两侧编译同一份采源码，避免把编译器差异算进内核。
2. **冷启动受控 A/B**。内核“优化前/后”的对比尽量同一次上电、交替顺序、各测两遍 取均值——本板“启动后第一次大分配明显更快”，不交替就会得出假结论。
3. **探针要撤**。 热路径分段计时每次系统调用多 12 次取时间戳（约 +1.2 μs）。 对照数字必须取撤掉探针之后的镜像。
4. **缺口径的要标注**。 例如“端到端启动”与“内核段启动”是两个不同的锚点， 报数时必须写清是哪一段（第 7 章）。

9.4　验收判据

每一处改动上线前都过同一组门：

层	判据	本册对应
宿主单测	文件系统与内存相关 crate 的宿主回归全绿（早期基线 237 项，收尾时内核 crate 261 项）	所有改动
板端功能	标准 harness 全项 + 功能用例（`ls`/`head`/`grep`/`sh -c`/退出码）	所有改动
数据正确性	同一文件多种分块读的校验和一致，且与主机侧直接导出的文件逐字节相同	第 4、5 章
内存不泄漏	mmap/touch/munmap 反复折腾后可用内存稳定；预取窗口的膨胀验证见第 4 章	第 4 章
一致性	TLB 一致性自检不匹配页数为 0（改 TLB 语义的改动必须跑）	第 5 章
压力	多二进制 `exec` 压力、`fork`/`exec` 风暴、`LEAKCHK` 与调度探针全过	第 2、3 章
可回退	动语义的改动都留运行期回退开关（如 `/proc/icache_flush`、`/proc/fault_around`）	第 2、4 章

表 47: 验收门。凡是一次改动动到了对外语义（指令缓存、TLB、访问时间、地址空间退休），都额外做了对应的专项自检

9.5　与主报告的关系

本分册的所有章节都可以直接并入主报告的“系统层：内核优化与系统能力”部分：章内 不再需要改写，编号由 Typst 自动生成。合并时需要一并迁移的是：

- `kernel_opt/numbers.typ` 里的数字宏（去重后并入 `data/numbers.typ`）；
- `kernel_opt/figures/` 下的七张图（主报告用 `#figph()` 引用，需按主报告 的图号方案重新编号）；

- `kernel_opt/_lib.typ` 的 `kfig()`（并入后可改用主报告的 `#figph()`）。